

Script Comparison: Windows 11 Image-Builder Feature Matrix

Comparison of four Windows-11 image-customization tools, plus a fifth reference (Rufus) consulted specifically for its `unattend.xml` generation techniques:

Tool	Location	Role
asl-win11	asl-win11maker.ps1 + asl-win11.functions.ps1 (repo root)	This project — modular, actively maintained
ntdevlabs	reference/tiny11builder-ntdevlabs/tiny11maker.ps1	Upstream original this project forked from
tiny11-automated (headless)	reference/tiny11-automated/scripts/tiny11maker-headless.ps1	CI/CD-oriented fork of ntdevlabs
winutil	reference/winutil/functions/private/Invoke-WinUtilISOScript.ps1	ISO-customization step embedded in a larger toolkit (ChrisTitusTech WinUtil)
Rufus (reference only)	reference/rufus/src/wue.c	USB-creation tool; not a WIM-servicing pipeline, consulted only for <code>unattend.xml</code> techniques (§6)

All four scripts are git submodules/files kept under `reference/` or this repo purely for comparison — see `CLAUDE.md` for the "reference-only, do not edit" policy.

1. Four-way feature matrix

Status note: every §4 integration below has now been ported into asl-win11 (as of the combined-builder work — driver injection, namespace-aware answer-file editing, edition enforcement, `support\` removal, the 6 extra scheduled tasks, the merged WU/telemetry tweak set, a selectable hardware-bypass strategy, and top-level error handling). The **asl-win11 column below is left as originally written**, describing the pre-combined-builder state, so this table still accurately documents what was learned from each reference tool. For asl-win11's *current* state, see `docs/tweak-catalog.md` and `CLAUDE.md` instead.

Feature	asl-win11 (pre-combined-builder)	ntdevlabs	tiny11-automated (headless)	winutil
Driver injection	❌ none	❌ none	❌ none	✅ Export-WindowsDriver -Online + DISM /Add-Driver /Recurse into install.wim and boot.wim index 2
autounattend.xml handling	Static copy; no XML parsing	Static copy; no XML parsing	Static copy; no XML parsing	✅ Namespace-aware DOM edit (ConvertTo-WinUtilISOAnswerFile) — inj / IMAGE/INDEX , strips empty p keys
Image index injected into XML	❌	❌	❌	✅ (see above)
Setup script pre-staging from answer file	❌	❌	❌	✅ Parses <sg:File> extension nodes, writes referenced scripts the image with correct encoding extension
Edition enforcement (ei.cfg / PID.txt)	❌	❌	❌	✅ Writes sources\ei.cfg , d sources\PID.txt
ISO support\ folder removal	❌	❌	❌	✅
Scheduled tasks removed	5 (see §1a)	5 (identical set)	5 (identical set)	11 (same 5 + 6 more WU-related §1a)
App package removal list	49 active prefixes (52 incl. 3 commented out)	~46 prefixes	57 prefixes (largest; adds AI/Recall/Copilot-era packages)	19 prefixes (smallest)
WU/telemetry registry tweaks	Broad (telemetry, sponsored apps, Copilot, Dev Home/Outlook auto-install, new Outlook, Teams) — see asl-win11.functions.ps1 Disable-* functions	Same tweaks as asl-win11 (asl-win11 is a modularized fork of this)	Same core set plus DiagTrack / WerSvc / PcaSvc / SysMain service disabling, WindowsAI / Recall keys, DoNotShowFeedbackNotifications	Different tweak set — AU\AUOpt UseWUSever , WUSever=http://localhost DODownloadMode=0 , BITS/UsoSvc/WaaSMedicSvc se disables (see §2 of the original comparison, retained below)
Hardware bypass registry tweaks	LabConfig 5-key bypass + AllowUpgradesWithUnsupportedTPMOrCPU + notification-cache suppression; install.wim only by default (Set-BypassHardwareChecks call is commented out in the pipeline, but Set-BootImageBypassTweaks always applies it to boot.wim)	Identical key set, applied to both install.wim and boot.wim unconditionally	Identical key set, applied to both install.wim and boot.wim unconditionally	No LabConfig-style offline bypass; relies on driver injection + (in the Rufus-style approach) unattend-embedded reg add command; instead
Boot.wim customization	Mounts index 2, applies hardware-bypass tweaks only	Mounts index 2, applies hardware-bypass tweaks only	Mounts index 2, applies hardware-bypass tweaks only	Injects drivers into boot.wim inc no hardware-bypass registry twe
End-to-end workflow vs. processing step	Full pipeline (ESD convert → mount → strip → tweak → export → ISO)	Full pipeline (monolithic)	Full pipeline (monolithic, CI-oriented)	Processing step only — recei already-mounted image path/pa from the caller
Logging	Start-Transcript to logs/	Start-Transcript to script root	Custom Write-Log (timestamped, leveled INFO/WARN/ERROR, dual console+file) plus machine-readable tiny11-buildinfo.json	Pluggable -Log scriptblock par (caller supplies logging, e.g. Wir own GUI)
Error handling / retry logic	try/catch around a few IsReadOnly/registry ops; no top-level trap	Same minimal pattern; manual re-check-and-remove at final cleanup only	\$ErrorActionPreference='Stop' + Set-StrictMode ; single top-level try/catch with emergency Dismount-WindowsImage -Discard + hive-unload on any failure; explicit exit 0 / exit 1	Caller's responsibility (it's a func not a standalone script)
OneDrive removal	✅ Remove-OneDriveSetup (takeown/icacls + delete OneDriveSetup.exe)	✅ same technique	✅ same technique + SysWOW64 copy + Run-key removal (HKLM and NTUSER)	❌ not implemented
Edge removal	✅ Remove-Edge (files + optional WebView2 + uninstall registry keys)	✅ same technique, always removes WebView2	✅ same technique + WinSxS microsoft-edge-webview package removal	❌ not implemented
ESD→WIM conversion	✅ prompts for index, Export-WindowsImage -CompressionType Maximum -CheckIntegrity	✅ same	✅ same, non-interactive (index from -INDEX param, validated against ESD image list)	N/A (processing step, assumes V already prepared)

Feature	asl-win11 (pre-combined-builder)	ntdevlabs	tiny11-automated (headless)	winutil
DISM cleanup + recovery export	✓ /Cleanup-Image /StartComponentCleanup /ResetBase + /Compress:recovery export pass (install.wim only)	✓ same	✓ same (Optimize-WindowsImage + Dismount-AndExport)	N/A (caller's responsibility)

1a. Scheduled tasks removed, by script

asl-win11 / ntdevlabs / tiny11-automated (identical 5-task set):

- Microsoft\Windows\Application Experience\Microsoft Compatibility Appraiser
- Microsoft\Windows\Customer Experience Improvement Program (whole folder)
- Microsoft\Windows\Application Experience\ProgramDataUpdater
- Microsoft\Windows\Chkdsk\Proxy
- Microsoft\Windows\Windows Error Reporting\QueueReporting

winutil (same 5, plus 6 more WU-specific, all recursive):

- Microsoft\Windows\InstallService
- Microsoft\Windows\UpdateOrchestrator
- Microsoft\Windows\UpdateAssistant
- Microsoft\Windows\WaaSMedic
- Microsoft\Windows\WindowsUpdate
- Microsoft\WindowsUpdate

2. Architecture comparison

Tool	Style	Notes
asl-win11	Modular — thin orchestrator (asl-win11maker.ps1) dot-sources a function library (asl-win11.functions.ps1) and calls a linear, annotated (REQUIRED / OPTIONAL) pipeline. Two arrays (\$appPackagePrefixes , \$scheduledTaskPaths) are the primary customization surface.	Easiest of the four to extend safely — new tweaks are a function + one annotated call.
ntdevlabs	Monolithic — single 535-line script, top to bottom, only two tiny helper functions (Set-RegistryValue / Remove-RegistryValue). Everything else is inline sequential code.	This is the upstream asl-win11 was forked and modularized from — explains the near-identical tweak set.
tiny11-automated (headless)	Monolithic — single 1003-line script, but organized as a flat set of ~30 function blocks followed by one top-level try/catch driver. No external modules; CI config (config/matrix_config.json) lives at the workflow level, not inside the script.	CI/CD-ready: fully parameterized (-ISO , -INDEX , -SCRATCH , -SkipCleanup), no interactive prompts, fails fast via Test-Prerequisites , emits tiny11-buildinfo.json for downstream workflow steps, has a Pester test (Test-BuildNumberParsing.Tests.ps1) covering its build-number parsing logic.
winutil	Plugin/integration point — a single function (Invoke-WinUtilISOScript) designed to be called from WinUtil's larger GUI toolkit. Takes an already-mounted image, a -Log scriptblock for caller-supplied logging, and switches like -InjectCurrentSystemDrivers / -InstallEditionId .	Not a standalone builder; it's the "customize a mounted image" step in someone else's pipeline. Cleanest interface for embedding, but owns none of the ISO lifecycle (copy/mount/export/burn).

3. Key gaps identified

What ntdevlabs has that asl-win11 lacks: Nothing feature-wise — asl-win11 is a modularized fork of ntdevlabs with an identical tweak/removal set, so there's no gap here. The only functional difference is architecture (modular vs. monolithic) and asl-win11's slightly larger app-removal list.

What tiny11-automated adds specifically for headless/CI use:

- Fully non-interactive parameter surface (-ISO , -INDEX , -SCRATCH , -SkipCleanup) — no Read-Host anywhere.
- Resolve-ImageIndex auto-corrects index drift instead of presenting a menu.
- Test-Prerequisites fails fast (non-admin, missing media, low disk space) instead of interactive recovery.
- Machine-readable tiny11-buildinfo.json output (Write-BuildInfo) for CI to consume build metadata.
- Top-level try/catch with emergency dismount/hive-unload and explicit exit 0 / exit 1 codes for CI pass/fail signaling.
- Dual console+file structured logging (Write-Log with levels) vs. plain transcript.
- A larger, more current app-removal list (57 prefixes, including AI/Recall/Copilot-era packages) and additional service disabling (DiagTrack , WerSvc , PcaSvc , SysMain).

Most comprehensive telemetry removal: tiny11-automated (headless) — same telemetry base as asl-win11/ntdevlabs, plus explicit service-level disabling (DiagTrack , WerSvc , PcaSvc , SysMain via Start=4) and dedicated Windows AI/Recall keys that don't exist in the

other three.

Best edge-case handling (ESD conversion, recovery export): asl-win11, ntdevlabs, and tiny11-automated all implement the same core ESD→WIM conversion and `/Compress:recovery export` pass; tiny11-automated is the most *robust* of the three because of its fail-fast prerequisite checks and emergency-cleanup try/catch — a corrupted run is far less likely to leave a dangling image mount than the other two, which have only ad-hoc retry-on-cleanup logic. winutil doesn't participate in this category at all since it assumes a WIM is already prepared.

4. Recommended integrations for asl-win11 (prioritized by impact)

All eight items below are now implemented as part of the combined-builder work. Left in place as the original prioritized rationale for *why* each was ported, with a  note on how it landed. See `docs/tweak-catalog.md` for the tweak-level detail and `CLAUDE.md` for the architecture.

-  **Implemented** as `Invoke-Tiny11EmergencyCleanup` + a top-level try/catch in `asl-win11maker.ps1`. **Top-level error handling with emergency cleanup** (*High impact, low effort*) — Adopt tiny11-automated's pattern: `$ErrorActionPreference = 'Stop'`, a single top-level try/catch around the pipeline in `asl-win11maker.ps1`, and an emergency handler that discards any still-mounted images (`Get-WindowsImage -Mounted | Dismount-WindowsImage -Discard`) and unloads registry hives before exiting. Today a mid-pipeline failure can leave `install.wim/boot.wim` mounted and hives loaded, blocking the next run.
-  **Implemented** as `ConvertTo-Tiny11AnswerFile`, called from `Initialize-PreparedAnswerFile`. **autounattend.xml image-index injection** (*High impact, medium effort*) — Adopt winutil's `ConvertTo-WinUtilISOAnswerFile` approach: namespace-aware injection of `/IMAGE/INDEX` into the answer file's `<MetaData>` so Setup installs the edition actually selected via `Resolve-InstallImageIndex`, instead of relying on the answer file's own (possibly stale) assumptions. Directly fixes a correctness gap shared by all three PowerShell scripts.
-  **Implemented** as `Set-Tiny11EditionConfig`. **ei.cfg / PID.txt edition enforcement** (*Medium impact, low effort*) — Add winutil's `Write-WinUtilISOEditionConfig` -style step: write `sources\ei.cfg` pinning the selected edition and delete `sources\PID.txt` so Setup doesn't fall back to a stale firmware product key. Cheap, and prevents an edition-mismatch failure mode none of the three PowerShell scripts currently guard against.
-  **Implemented** as `Remove-IsoSupportFolder`. **support\ folder removal from the ISO** (*Low impact, trivial effort*) — One-line size reduction, already proven safe by winutil.
-  **Implemented**, but shipped **commented out** rather than active — see `docs/tweak-catalog.md` §2. **Additional WU-related scheduled task removals** (*Medium impact, trivial effort*) — Add winutil's 6 extra task paths (`InstallService`, `UpdateOrchestrator`, `UpdateAssistant`, `WaaSMedic`, `WindowsUpdate X2`) to `$scheduledTaskPaths` in `asl-win11maker.ps1` — pure additive config change, no new function needed.
-  **Implemented** as `Write-Log` / `Write-Phase` (structured logging) and `Write-BuildInfo` (JSON). **Structured/leveled logging + build-info JSON** (*Medium impact, medium effort*) — Borrow tiny11-automated's `Write-Log` (leveled, dual console+file) and `Write-BuildInfo` JSON emission. Useful if asl-win11 is ever driven from CI or wrapped by another tool; low risk since it's additive to the existing `Start-Transcript` call.
-  **Implemented** as `Export-HostSystemDrivers` + `Add-DriversToImage`, wired into both `install.wim` and `Update-BootImage` (`boot.wim`), gated by `-InjectSystemDrivers`. Also went further than originally scoped: added KVM/QEMU virtio-win driver injection (`Resolve-VirtioDriverSource`, `Add-VirtioDriversToImage`, `Install-VirtioGuestToolsAtFirstLogon`), since `boot.wim` needs the virtio storage drivers or Windows Setup can't see a VM's disk at all. **Driver injection (-InjectCurrentSystemDrivers -style)** (*High impact for hardware compatibility, higher effort*) — Port winutil's `Export-WindowsDriver -Online + DISM /Add-Driver /Recurse` into both `install.wim` and `boot.wim` index 2. Valuable for building images targeted at specific hardware, but it's the largest new function to write/test of anything in this list — lower priority than the correctness/robustness items above unless there's an immediate need for it.
-  **Implemented** as `Add-AnswerFileBypassCommands` (the Rufus/Unattend path) alongside the existing `Set-BypassHardwareChecks` (the Hive path), selectable via the new `-BypassMode` parameter (`None / Hive / Unattend / Both`; **default None** — a deliberate behavior change from always applying the bypass to `boot.wim`). `Enable-LocalAccountOOBE` already covers the `BypassNRO` half mentioned below. **Dual-path hardware-bypass strategy from Rufus (optional/defensive)** (*Low-medium impact, medium effort*) — Rufus's `wue.c` shows a fallback hierarchy worth knowing about even though asl-win11 already does direct offline-hive editing successfully: it *prefers* direct registry-hive mount/edit for the LabConfig bypass keys, and only falls back to embedding `reg add` commands in `RunSynchronousCommand` blocks inside `unattend.xml` when direct hive access isn't available (e.g., sandboxed/Store-distributed builds). Not urgent for asl-win11 today, but worth keeping in mind if this script is ever adapted to run in a more restricted environment. Rufus's other unattend.xml techniques worth knowing about even though they don't map to a direct ported feature: `BypassNRO + HideOnlineAccountScreens` for defeating the mandatory Microsoft-account OOBE screen (asl-win11 already does the `BypassNRO` half via `Enable-LocalAccountOOBE`), and the fact that Setup only auto-copies `Autounattend.xml` into `%WINDIR%\Panther\unattend.xml` when a windowsPE pass is present — a quirk worth remembering if asl-win11's autounattend.xml handling is ever reworked (item 2 above).

Not recommended for porting: setup-script pre-staging from `<sg:File>` extension nodes (winutil) — no current asl-win11 use case needs scripts injected via the answer file, and it adds XML-parsing complexity for a feature nothing in this pipeline currently calls for.

New candidates identified in the 2026-10-05 submodule refresh (not yet evaluated/implemented):

9. **tiny11-automated's WinRE staging/CompactOS fix** (`8bde4bd` , "suppress WinRE staging error 0x80070003 and enable compact OS install") — relevant to this repo's `-Core -only Remove-WindowsRecoveryEnvironment / Compress-WinSxS` functions, which do similar WinRE/WinSxS surgery. Worth reading before the next time either of those functions is touched, in case the same `0x80070003` failure mode applies here.
10. **winutil's storage-driver injection fix** (`abcb23` , "fix: inject Setup storage into boot.wim") — **a likely correctness gap in this repo's own driver injection**, not just an FYI. winutil dropped its filename-based storage-driver fallback heuristic (`iaahci|iastor|vmd|irst|rst`) after finding it let non-storage "RST companion" INFs reach Setup, and now *also* injects SCSIAdapter/HDC driver packages directly into boot.wim index 2 via DISM, rather than relying on `$WinpeDriver$` staging alone. This repo's `Test-StorageDriverInf` still carries the same filename-fallback pattern (extended with `viostor|vioscsi|nvme` for virtio), and `Add-WinPEStorageDrivers` still never touches boot.wim. See `docs/build-options-reference.md` 's Drivers section for the full writeup and what re-mounting boot.wim for this would cost against the current design's stated rationale (`CLAUDE.md` 's `Add-WinPEStorageDrivers` note, KB2686316).

Appendix: original winutil vs. tiny11maker baseline comparison

(Retained as the original two-way analysis this document was built from; paths updated to reflect the current `asl-win11maker.ps1` / `asl-win11.functions.ps1` naming and the `reference/winutil/` submodule location.)

What winutil has that asl-win11 LACKS

1. Driver Injection (`-InjectCurrentSystemDrivers`)

- Exports all drivers from the running system via `Export-WindowsDriver -Online`
- Injects into install.wim via DISM `/Add-Driver /Recurse`
- Also injects into boot.wim index 2 (so setup PE has the same drivers)
- Temp directory created/cleaned up automatically
- **Verdict: Significant missing feature** — valuable for hardware compatibility

2. autounattend.xml XML Manipulation

- `ConvertTo-WinUtilISOAnswerFile` : namespace-aware XML processing that injects the `IMAGE/INDEX` metadata into the answer file so Windows Setup installs the correct edition
- Removes empty/invalid product keys from the XML before embedding
- `Get-WinUtilISOScriptChildElement` : helper for namespace-safe element creation
- **Verdict: asl-win11 just copies autounattend.xml as-is; it does not inject the image index into the XML**

3. Setup Script Pre-staging from autounattend.xml

- Parses `<sg:File>` extension nodes in autounattend.xml
- Writes each referenced script to its Windows path under the mount dir with correct encoding:
 - `.ps1` , `.xml` → UTF-8
 - `.reg` , `.vbs` , `.js` → UTF-16 LE with BOM
- Ensures scripts survive Windows Setup stripping unrecognized XML namespaces
- **Verdict: Currently absent from asl-win11**

4. Edition Enforcement (`-InstallEditionId`)

- `Write-WinUtilISOEditionConfig` : writes `sources\ei.cfg` to pin the edition
- Deletes `sources\PID.txt` to prevent setup from forcing a stale firmware product key
- **Verdict: asl-win11 doesn't write ei.cfg or remove PID.txt**

5. ISO `support\` Folder Removal

- Removes the `support\` folder from the ISO contents directory
- Reduces final ISO size
- **Verdict: Not done in asl-win11**

6. Additional Scheduled Task Removals (6 more than asl-win11's 5)

winutil removes these task paths that asl-win11 skips:

- `Microsoft\Windows\InstallService` (recursive)
- `Microsoft\Windows\UpdateOrchestrator` (recursive)
- `Microsoft\Windows\UpdateAssistant` (recursive)
- `Microsoft\Windows\WaaSMedic` (recursive)
- `Microsoft\Windows\WindowsUpdate` (recursive)
- `Microsoft\WindowsUpdate` (recursive)
- **Verdict: More thorough WU task cleanup**

7. Additional Windows Update Registry Tweaks

winutil applies these that asl-win11 omits:

Registry Path	Value	Note
...WindowsUpdate\AU\AUOptions	1	Disable auto-update options
...WindowsUpdate\AU\UseWUServer	1	Force WSUS server policy
...WindowsUpdate\DisableWindowsUpdateAccess	1	Block WU access policy
WUServer	http://localhost:8080	More specific localhost URL
WUStatusServer	http://localhost:8080	Same
...UScheduler_Oobe\WindowsUpdate\workCompleted	1	Block WU OOBE install
...DeliveryOptimization\Config\DODownloadMode	0	Disable delivery optimization
Services\BITS\Start	4 (disabled)	Background Intelligent Transfer
Services\UsSvc\Start	4 (disabled)	Update Orchestrator Service
Services\WaaSMedicSvc\Start	4 (disabled)	WaaS Medic Service

asl-win11 instead **deletes** the WaaSMedicSVC and UsSvc keys entirely (more aggressive) and sets `DisableOnline = 1` on OOBE (which winutil doesn't do).

8. Pluggable Logger (`-Log` scriptblock parameter)

- winutil accepts a custom logging function
- Enables integration into larger workflows (e.g., GUI progress display in WinUtil itself)
- **Verdict: asl-win11 always writes to transcript/stdout**

What asl-win11 HAS that winutil LACKS

1. Full End-to-End ISO Build Workflow

winutil is a *processing step only* — it receives an already-mounted image path. asl-win11 owns the full pipeline:

- ESD → WIM conversion
- Interactive image index selection
- `Copy-SourceImageFiles` (ISO copy to scratch)
- `Mount-InstallImage`, `Show-ImageMetadata`
- `Complete-InstallImage` (DISM cleanup + recovery export)
- `Set-BootImageBypassTweaks` (boot.wim hardware bypass)
- `New-Tiny11Iso` (oscdimg ISO creation)
- `Invoke-Tiny11Cleanup` (eject, delete temps, retry logic)
- `Confirm-AutounattendXml` (download from GitHub if missing)

2. Edge Removal (File + Registry)

- `Remove-Edge` : removes Edge browser dirs, EdgeUpdate, EdgeCore
- Optional WebView2 removal (`-SkipWebView` switch)
- Cleans up Edge/EdgeUpdate uninstall registry entries
- **Not implemented in winutil**

3. OneDrive Setup Removal

- `Remove-OneDriveSetup` : removes `Windows\System32\OneDriveSetup.exe`
- Uses `takeown / icacls` for permission elevation
- **Not implemented in winutil**

4. Hardware Bypass in Boot.wim (`Set-BootImageBypassTweaks`)

- Mounts boot.wim index 2, loads hives, applies LabConfig bypass keys, saves
- winutil's driver injection touches boot.wim, but does **not** apply hardware bypass registry tweaks to it
- **Verdict: asl-win11 applies bypass to both install.wim and the setup PE**

5. Much Larger App Removal List (49 vs 19 packages)

asl-win11 removes 30+ additional package prefixes beyond winutil's 19, including:

- All Xbox app family (TCUI, App, GameOverlay, GamingOverlay, IdentityProvider, SpeechToText)
- Cortana (`Microsoft.549981C3F5F10`)
- Dolby OEM apps, Intel Management Status

- Mixed Reality Portal
- SkypeApp, People, Phone Link, WindowsCamera, WindowsAlarms, WindowsMaps
- Mail/Calendar (windowscommunicationsapps), OneNote UWP, OfficePushNotification
- Microsoft Family Safety, 3D Viewer, Wallet, Sticky Notes, ZuneVideo

winutil has `Microsoft.Paint` which asl-win11 currently has commented out.

6. DISM Image Cleanup Step

- `Complete-InstallImage` runs `/Cleanup-Image /StartComponentCleanup /ResetBase` before export
- Significantly reduces final image size
- **Not done in winutil (caller's responsibility)**

7. OOBE-Specific WU Suppression

- Sets `HKLM\zSOFTWARE\Microsoft\Windows\CurrentVersion\OOBE\DisableOnline = 1`
- Uses RunOnce commands to stop/disable WU service on first boot post-OOBE
- Fully deletes WaaSMedicSVC and UsoSvc service keys (vs winutil setting Start=4)
- `DoNotConnectToWindowsUpdateInternetLocations = 1` policy key